

Deadstock Management System

Prototype Stage Report for UCS503P

Submitted to: Dr. Asif

Thapar Institute of Engineering and Technology

Aryan Gupta, CSED*

Abhilakshya Puri, CSED[†]

Farhan Kansal, CSED[‡]

September 13, 2026

1 Higher-order goal

This project reduces capital wastage in retail supply chains by giving unsold (“deadstock”) inventory a structured, trustworthy path back into the market. The prototype built so far connects vendor inventory logging, date-driven lifecycle automation, a rule-based verification engine, and a verified public marketplace behind a live React and Express stack. The wider transaction, payments, and analytics capabilities described later remain proposed extensions for the final submission stage.

2 Time-to-value

- We delivered the core *list, verify, publish, purchase-ready* loop end to end before layering on payments or analytics.
- We iterated weekly, starting with authentication and inventory CRUD before adding verification and marketplace publishing.
- Success in this iteration is a working prototype where a vendor can log stock, have it auto-flagged, get it verified, and see it live in the marketplace, all backed by 24 automated tests.

3 Problem Statement

An estimated 20–30% of retail inventory value sits idle as slow-moving or dead stock annually, and vendors currently have no structured way to recover this trapped capital. Common pain points include:

- **No real-time visibility:** vendors track stock manually, so items are only noticed *after* they have become deadstock.
- **No way to verify deadstock:** buyers cannot trust a listing without proof of condition, quantity, or authenticity, since none of this is independently confirmed today.

*Roll No: 1024030455, Email: agupta_be24@thapar.edu

[†]Roll No: 1024030445, Email: apuri_be24@thapar.edu

[‡]Roll No: 1024030451, Email: fkansal_be24@thapar.edu

- **No resale marketplace:** even genuinely resellable stock has no dedicated channel connecting vendors with willing buyers.
- **Compliance risk:** regulated categories (medical, pharmaceutical, food, alcohol, tobacco) cannot resell freely; any resale channel must route them through review.

Why this matters now: unsold stock is routinely discarded or discounted below cost, a direct loss to the vendor, while a trusted resale channel with an auditable provenance trail could recover that value instead.

4 Proposed Solution

4.1 Overview

A **web-based verified marketplace** where vendors track, verify, and resell deadstock inventory to other buyers, built from the following components: a vendor inventory dashboard with automated aging alerts, a rule-based verification layer, verified marketplace listings, and compliance guardrails for regulated categories.

4.2 Core workflow

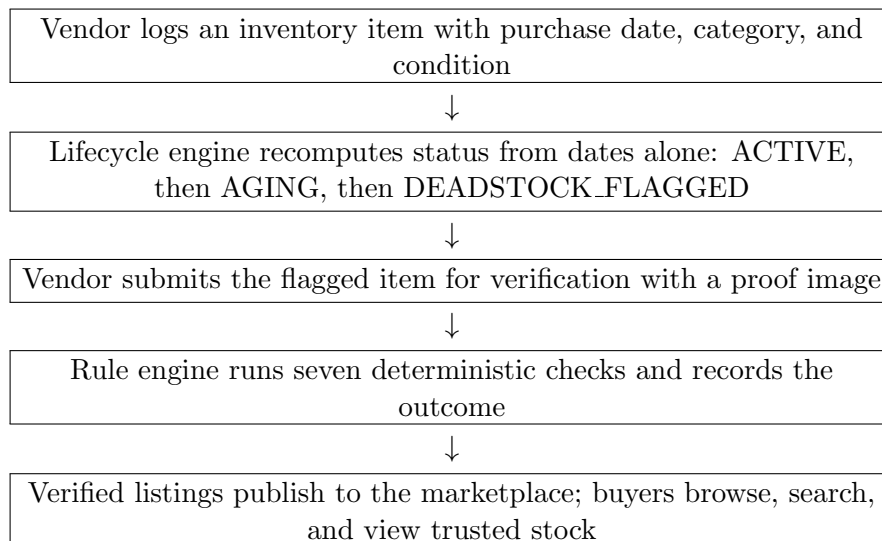


Figure 1: End-to-end workflow of the current prototype.

4.3 Current prototype and implementation status

The implemented prototype runs the full loop above against a live Express API and a React vendor portal. In demo mode the backend uses an in-memory data store instead of PostgreSQL, so no database setup is required for a live walkthrough; restarting the API resets state to the seeded demo accounts. Proof upload uses an in-browser data URL rather than real object storage in this mode. Payments, purchase transactions, and analytics are not yet implemented and remain future work, as detailed in Section 9.

5 Solution Approach

This is an engineering course project, so the emphasis is on a working, testable marketplace and workflow rather than novel algorithm research.

5.1 Verification engine

Verification is checklist- and rule-based, not ML-driven. Every submission is checked against seven deterministic rules: product name, category, quantity, condition, proof presence, dead-stock eligibility, and regulated category membership.

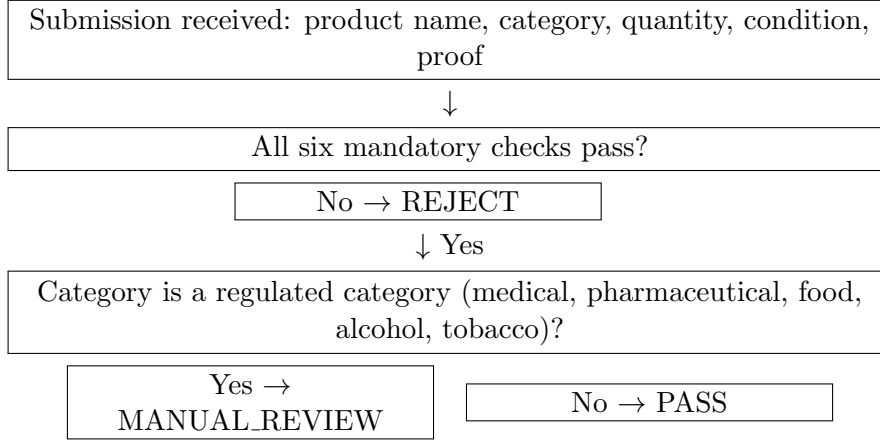


Figure 2: Implemented rule-based verification decision flow.

A failure on any mandatory check rejects the submission outright; a pass on all six but membership in a regulated category escalates to a manual review queue instead of auto-publishing. Every outcome, including the full rule result, is stored on the verification record so a reviewer can see exactly why an item passed, failed, or was escalated.

5.2 Web architecture

A 3-tier web architecture, implemented and running:

- **Frontend:** React 18, Vite, TypeScript, and Tailwind CSS, covering the vendor dashboard, inventory CRUD, admin review queue, and the public buyer marketplace.
- **Backend API:** Node.js and Express with JWT authentication, bcrypt password hashing, Zod validation, the inventory lifecycle engine, the verification rule engine, and listing policy enforcement.
- **Database:** PostgreSQL via Prisma ORM, modelling users, inventory items, verification records, and listings (an in-memory store substitutes for this in prototype demo mode).

5.3 CI/CD and current test status

- A GitHub Actions workflow installs dependencies, typechecks, tests, and builds the backend on every push.
- The backend suite currently passes 18 of 18 tests (Vitest and Supertest) covering authentication, the aging and deadstock lifecycle, the rule engine, listing policy, and the verification and marketplace API together.
- The frontend suite currently passes 6 of 6 tests (React Testing Library) covering status-to-label mapping and inventory table behaviour.
- Continuous deployment to a staging environment is not yet wired up and remains a subsequent deliverable.

6 Evaluation Criterion

6.1 Primary evaluation metric

Time-to-listing (TTL): time between a vendor's item being flagged as deadstock and it appearing as a verified, live marketplace listing.

- Target: median TTL ≤ 24 hours during pilot window.
- Attribution: the schema already stamps every stage (`deadstock_flagged_at`, `submitted_at`, `reviewed_at`, `published_at`), so TTL is directly computable from stored timestamps once a pilot is run.

6.2 Secondary evaluation metrics

- **Recovery rate:** percentage of listed deadstock items sold within the pilot window (requires the transaction flow in Section 9.2).
- **Verification pass rate:** percentage of submitted listings passing rule checks without manual escalation, already observable from stored `rule_result` data.
- **Buyer trust:** buyer-reported confidence in listing accuracy, planned for pilot feedback collection.
- **Reliability:** the automated test suite and CI pipeline are the current proxy for reliability ahead of a staging deployment.

6.3 Pilot validation plan

- Recruit a small group of pilot vendors and buyers once persistence and deployment (Section 9.2) are in place.
- Compare recovery rate and time-to-listing before and after within the iteration window, using the timestamp fields already present in the schema.

7 Scalability

1. **Scalable (theoretically):** the frontend and backend are already decoupled, the API is stateless aside from the JWT token, and Prisma indexes are defined on the hot query paths (`vendor_id`, `status`, `verification_status`, `submitted_at`, `listing_status`, `published_at`).
2. **Operationally deployable (practically):** the system targets common web hosting: a managed PostgreSQL database (Supabase-compatible), a containerized or platform-hosted deployment, and the existing CI pipeline extended with a staging target.

8 Engine availability heuristic

The prototype already uses a mature set of tools rather than inventing new infrastructure: Express as the web framework, Prisma and PostgreSQL for persistence, JWT and bcrypt for authentication, Zod for validation, React and Vite for the frontend, and Vitest, Supertest, and React Testing Library for automated testing, with GitHub Actions as the CI runner. This lets the team focus on marketplace design, verification correctness, and measurement rather than infrastructure.

9 Project scope and deliverables

9.1 Initial deliverable status

- Vendor inventory entry and aging alert logic: **done**.
- Listing creation with proof upload: **done** (data URL fallback in demo mode; real object storage pending).
- Rule-based verification checklist, including regulated-category escalation: **done**.
- Buyer-facing browse and search page for live listings: **done**.
- Timestamps for TTL measurement: **done** in schema; not yet measured against real pilot data.
- CI: build, backend unit tests, and typecheck: **done**; linting is not yet configured.
- CD to staging: **not yet started**.

9.2 Subsequent deliverables

- Persist to PostgreSQL/Supabase and real object storage for proof uploads, replacing the in-memory demo store.
- Transaction and purchase flow between buyer and vendor.
- Buyer rating and feedback, and a decision on whether it lives inside verification records or a separate store.
- Vendor and admin dashboards for recovery-rate metrics.
- Extend automated test coverage to the transaction flow once implemented.

10 Risks and mitigations

- **Regulatory risk:** regulated categories never auto-publish; they are routed to manual compliance review, already enforced by the rule engine.
- **Trust/fraud risk:** proof is required at submission time and every verification decision is stored with its full rule result for audit.
- **Vendor adoption:** inventory logging and aging alerts are already minimal-step, surfaced directly on the dashboard.
- **Evaluation measurement ambiguity:** lifecycle and verification timestamps were defined in the schema before development began, so TTL is measurable without retrofitting.

11 Summary

The prototype delivers a working list, verify, publish, purchase-ready loop on a live Express API and React frontend, with a deterministic rule-based verification engine, automated lifecycle management, and 24 passing automated tests gated by continuous integration. It remains focused on engineering workflow, trust, and scalability readiness rather than research-driven novelty, and the next iteration's priority is persistence, the transaction flow, and a staged deployment so the already-instrumented time-to-listing metric can be measured against real pilot data.